CSS Day 4 — Layout

Detailed theory notes for internship learning and revision.

1. CSS Box Model

Every HTML element can be understood as a box: **content, padding, border, margin**. Content is the actual material; padding is space inside the border; border is the boundary; margin is space outside the border. **box-sizing: content-box** generally applies width/height to content, while **border-box** includes padding and border in the declared dimensions.

2. display Property

The display property controls how an element participates in layout. **block** generally starts a new line; **inline** stays in the line; **inline-block** combines inline flow with box dimensions; **none** removes the element from layout; **flex** creates a flex container; **grid** creates a grid container.

3. Positioning

static is the normal default flow. **relative** keeps the element's space while allowing offsets and can establish a containing block for positioned descendants. **absolute** removes the element from normal flow and positions it relative to an appropriate containing block. **fixed** positions relative to the viewport. **sticky** behaves normally until a scrolling threshold is reached and can then stick within its scrolling context.

4. Flexbox Fundamentals

Flexbox is a one-dimensional CSS layout system for arranging items mainly in a row or column. **display: flex** makes a parent a flex container and its direct children flex items.

5. Flex Direction

flex-direction determines the main axis. **row** is the default horizontal arrangement; **column** is vertical; row-reverse and column-reverse reverse the order.

6. Main Axis and Cross Axis

The **main axis** follows flex-direction. The **cross axis** is perpendicular. With row, main is horizontal and cross is vertical. With column, main is vertical and cross is horizontal.

7. Flex Alignment and Spacing

justify-content distributes items along the main axis: flex-start, center, flex-end, space-between, space-around, and space-evenly. **align-items** aligns items on the cross axis: flex-start, center, flex-end, stretch, and baseline. **gap** creates spacing between flex items.

8. CSS Grid Basics

CSS Grid is a two-dimensional layout system using rows and columns. **display: grid** creates a grid container. **grid-template-columns** defines columns. The **fr** unit represents a fraction of available space. **gap** creates spacing between grid cells.

9. Flexbox vs Grid

Flexbox is primarily one-dimensional and is useful for rows or columns, such as navigation. **Grid** is two-dimensional and is useful for row-and-column layouts, such as cards. Memory aid: Flexbox = line; Grid = matrix.

10. Applying Day 4

Use Flexbox for navigation or arranging profile information. Use Grid for project/card layouts or form fields in columns. Use the box model for dimensions and spacing, and positioning only when a particular element needs controlled positioning.

11. Quick Revision

Box model = content → padding → border → margin. **display** controls layout behavior. **position** controls positioning. **flex** = one-dimensional layout. **justify-content** = main-axis distribution. **align-items** = cross-axis alignment. **gap** = spacing. **grid** = two-dimensional rows and columns.